Grumpy Gamer

Teaching AI to Master Classic Games

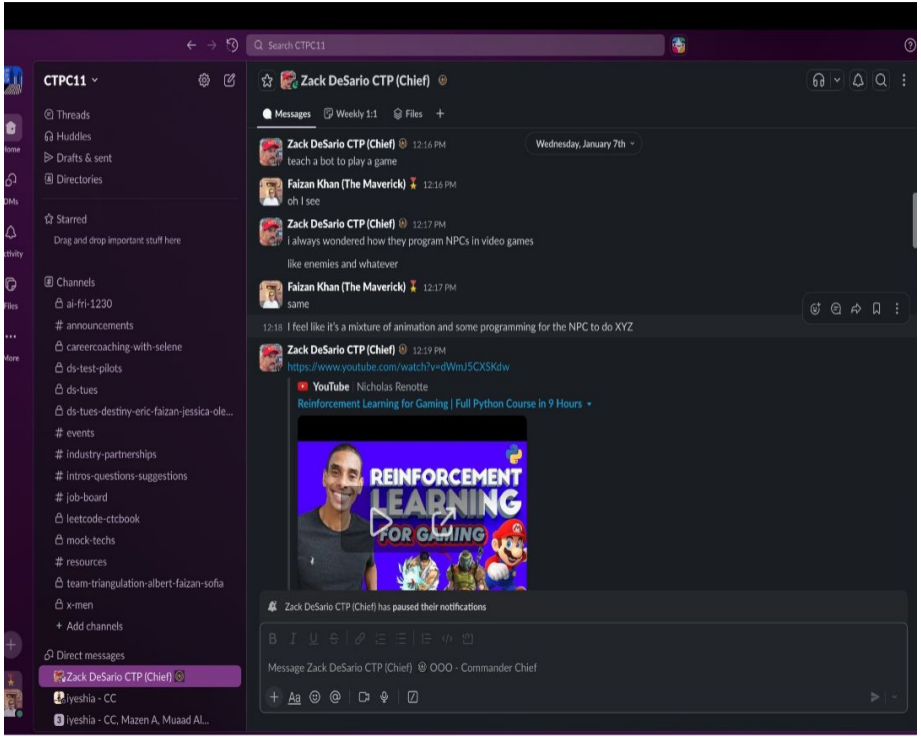
CISC 4900 - Senior Project | Spring 2026

Faizan Khan | Brooklyn College, CUNY

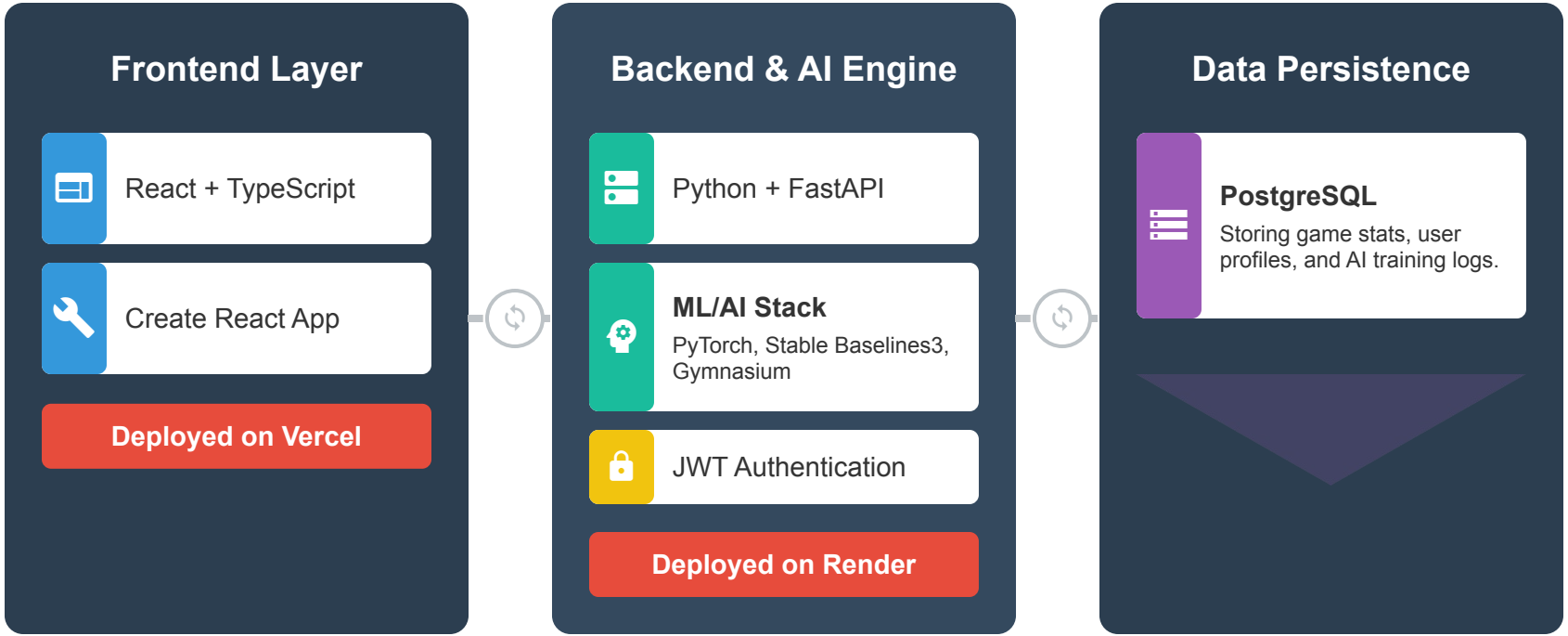
Supervisor: Zack Desario | CUNY Tech Prep

The Spark

It started with a simple question from my supervisor Zack: How do games program NPCs?” He shared a Reinforcement Learning course by Nicholas Renotte, and something clicked for me. Instead of programming NPCs in a game, what if I flipped the idea. What if I built classic games themselves, then trained AI agents to actually beat them?



Tech Stack



Hosted via Render

Team Roles and Collaboration

My Role (Faizan Khan)	Supervisor (Zack Desario)
	
Main Programmer & Solo Contributor	**Project Supervisor**
Pitched the core idea: Develop a web platform to explore if AI can beat classic games (Wordle, Sudoku).	Provided the initial project idea: research how games can program NPCs (Non-Player Characters)
Responsible for implementing game environments and developing all hybrid AI agents.	Shared resources on advanced topics, including Reinforcement Learning and Evolutionary Computing.

My Vision: Problem-to-Solution

- **The Problem Space:** Current platforms for puzzle games often feature simple, rule-based AI (Minimax or Heuristic Logic) that is predictable and offers no insight into modern machine learning. This leaves a gap and developers to interact with advanced Reinforcement Learning (RL) agents in a practical engaging context.
- **The Solution:** Develop “Grumpy Gamer” to act as a public-facing sandbox. It bridges the gap between complex PyTorch/Stable Baselines3 model and an intuitive web interface, allowing users to play against, and learn from, advanced AI strategies.
- **Value Proposition**: My contribution is creating a unified, scalable platform capable of hosting and benchmarking both classical and state-of-the-art RL agents, turning theoretical AI concepts into practical, interactive game scenarios.

My Role in the Project

- **Organizational Fit**: As a solo developer, I had to be the Project Manager, the Frontend Developer, the Backend Architect, the ML Engineer, and the QA team sometimes all in the same afternoon.
- **Contribution Focus**: My primary focus was on the technical implementation of the hybrid AI engine and creating the necessary infrastructure (FastAPI, PostgreSQL) to support real-time agent benchmarking.
- **Relationship to Supervisor**: Zack Desario acted as an academic mentor and advisor, offering conceptual guidance on Reinforcement Learning strategies and best practices for technical implementation, rather than acting as a traditional team lead or product owner.
- **Autonomy in Decision-Making:** All technology choices (React, PyTorch, FastAPI) and the definition of the Minimum Viable Product (MVP) were my responsibility, giving me complete autonomy over the project's technical direction.

The Initial Roadmap & Velocity

The project was executed across 15 estimated weekly sprints, structured into four strategic phases:

- **Phase 1: Foundation & Game Logic (Weeks 1-3):** Setup core architecture, initial game environment integration (MVP logic), and preliminary AI research/refactoring.
- **Phase 2: Hybrid AI Core Implementation (Weeks 4-8):** Focused on PPO/DQN implementation, advanced game training, strategy fine-tuning, and starting the Interactive Analytics Dashboard.
- **Phase 3: Frontend-End Refinement & QA (Weeks 9-12):** Built Spectator/Replay Mode, refined the UI/game feel, and conducted performance optimization and documentation.
- **Phase 4: Finalization & Handover (Weeks 13-15):** Final testing, deployment, refinement, and preparation for the final presentation and handover.

Velocity Tracking and Recordkeeping

- **Agile Methodology:** Tasks were managed via short sprints with an estimated commitment of 15 hours per week.
- **Documentation:** Progress is formally tracked using the Github Projects board, ensuring transparent recordkeeping of completed and pending tasks.
- **Project Board:** Full log of activity is available here: https://github.com/users/jellyfishing2346/projects/6

Critical Decision: Scope Management

Abandoned Feature: Full Chess Environment Integration

- **The Downscope:** I initially planned to fully integrate the Chess environment and train an RL agent.
- **Reason for Change:** The complexity of the state-action space in Chess presented a significant time sink for environment setup and required extensive training time to achieve competitive performance.
- **Decision:** I chose to delay the full Chess implementation. This freed up crucial time to focus on stabilizing the core PPO/DQN agents for the MVP games (Wordle, Connect Four) and refining the interactive analytic features.

Prioritized Feature: Agent-vs-Agent Spectator Mode

- **The Priority:** I shifted focus to completing the Spectator Mode functionality (Use Case 3), which allows two AI agents to play against each other.
- **Justification:** This feature is critical to the project's value proposition, as it provides a live, visual benchmarking tool that allows users (and instructors) to directly compare the performance of the complex RL agents versus the simple heuristic logic.
- **Status:** The infrastructure is now set up and functional, ensuring that the platform's core differentiation on AI comparison is robust.

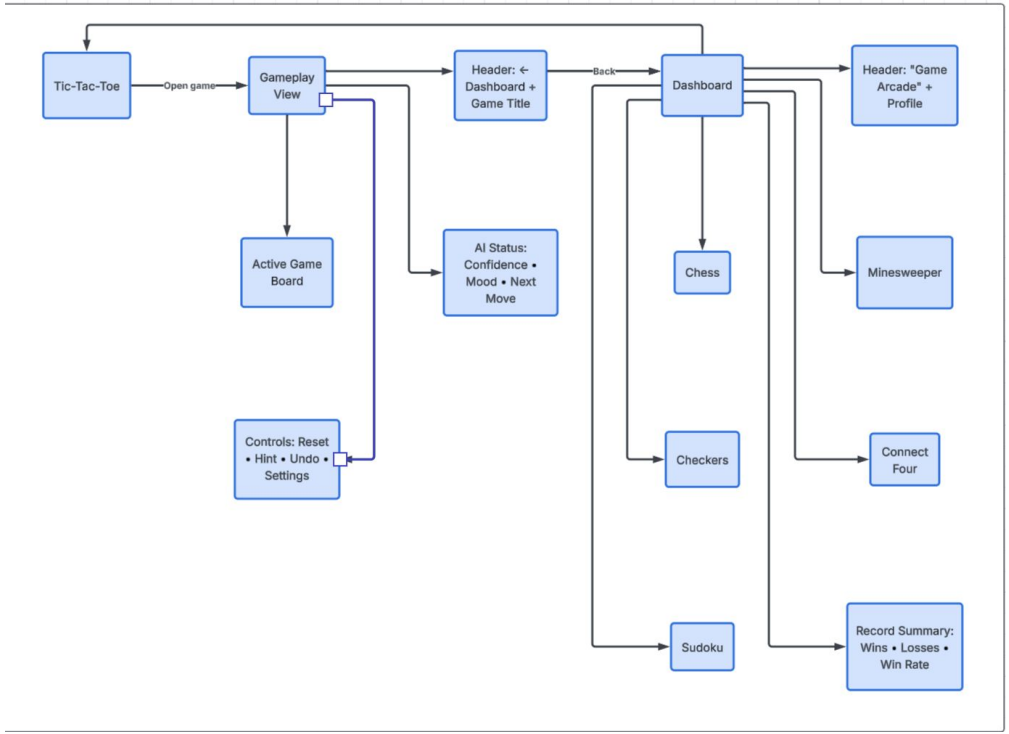
Design Evolution: Initial Sketch

- **The Concept:** My initial idea was a purely utilitarian “console application” look at a simple, dark interface focused entirely on the game board and a raw output of the AI agent’s chosen move.
- **Core Elements:** The original sketch focused only on three components: the **Game Environment** (a basic HTML canvas), a **Move Input Field**, and a **Terminal Output** for win/loss results.
- **Initial Scope:** The early design excluded features like user authentication, persistent databases, and complex analytics. It was strictly a proof-of-concept for the backend AI integration.
- **Pivot to UX:** I quickly realized that to fulfill the project’s value proposition (visualization and learning), a professional and engaging UI/UX was mandatory, leading to the low-fidelity wireframes in the next phase.

Design Evolution: Low-Fidelity Wireframes

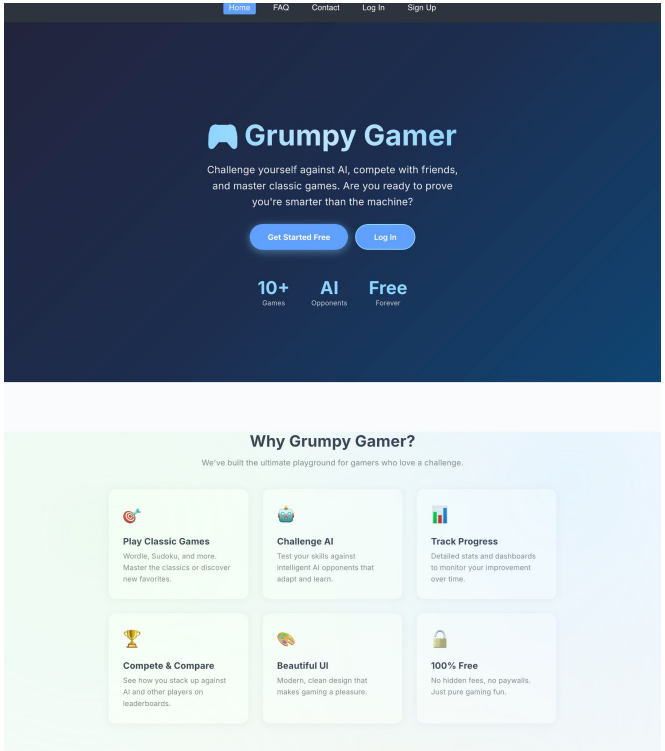
These wireframes visualize the layout for the platform's two core views, transitioning from the minimal "console application" concept to a user-friendly interface.

- **Dashboard:** This view displays cards for each game (e.g., Tic-Tac-Toe, Chess, Minesweeper), along with a summary of the user's win/loss record and overall progress.
- **Gameplay View:** This is the main interaction area, featuring the active game board, an "AI status" panel showing the model's confidence or current "mood", and critical user controls like "Reset" or "Hint".
- **Link**: [Gaming Dashboard UI Wireframe]



Design Evolution: High-Fidelity Mockup

- **Visual Identity:** The final design actualizes the “Grumpy Gamer” theme using a dark, high-contrast color palette with a single bright accent color (e.g., neon green or electric blue). This creates a competitive, professional atmosphere.
- **Refined Elements:** Key UI components are polished, transforming the functional wireframes into an engaging product. For example, the status panels use visual components like a **Confidence Meter** or **Decision Heatmaps** rather than simple text outputs.
- **Goal:** The high-fidelity design ensures that platform is not only functional but also intuitive and visually appealing, maximizing user engagement with the core AI-benchmarking features.



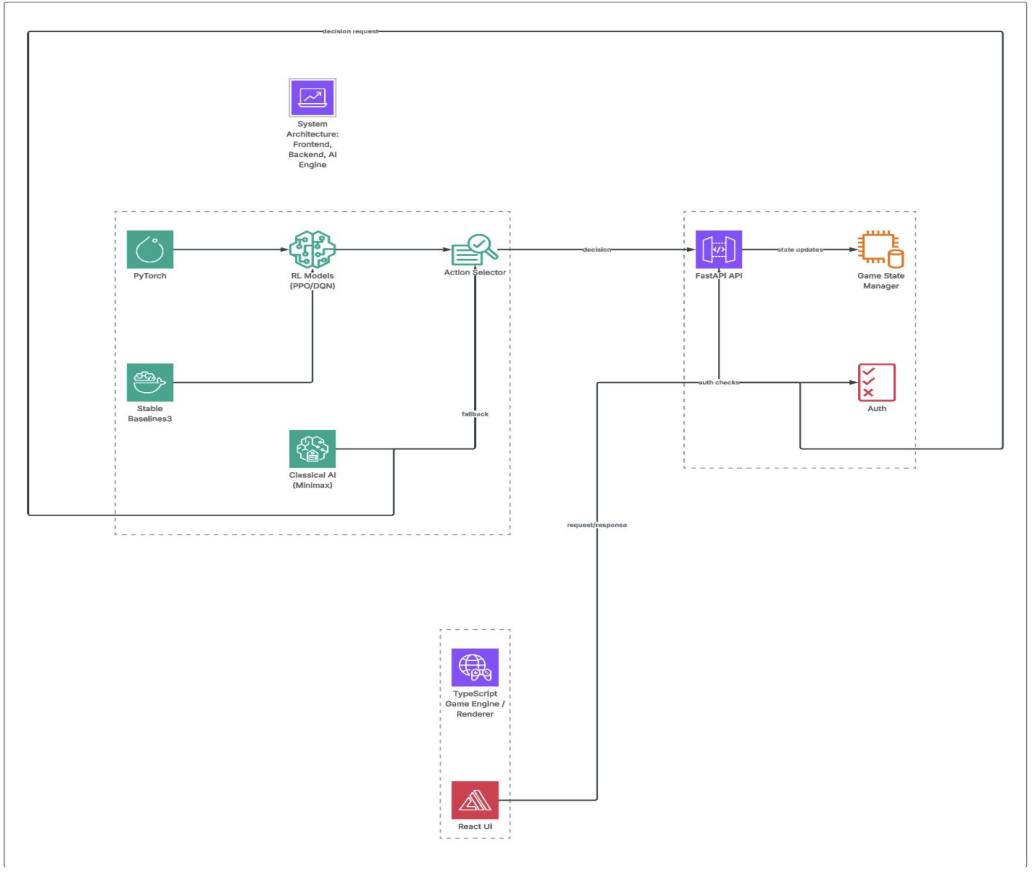
UI/UX & Design Choices

- **The “Grumpy Gamer” Theme**: The visual identity is built around a competitive, slightly aggressive theme, primarily using a dark, high-contrast palette (dark gray/black background). This design choice is intended to make the platform feel professional and focused on performance metrics, mirroring the seriousness of AI model benchmarking.
- **Color Palette (Accent Color):** A single, bright accent color (e.g., neon blue or electric green) is used sparingly to draw the user’s eye to critical areas like **“Play Now” Call-to-Action** buttons and key analytical data points (like win rates).
- **Driving the User:** The structure of the **Dashboard i**s designed to immediately present the user with a challenge (game cards and current stats), driving them toward the primary action of selecting a game.
- **Enhanced Gameplay UX:** Within the Gameplay View, critical information like the **“AI Status” panel** and **Confidence Meter** are prominent. This provides immediate feedback on the model’s decision-making, which is essential for the student learner use case.

System Architecture

This diagram illustrates the separation between your frontend, the API management, and the Reinforcement Learning logic.

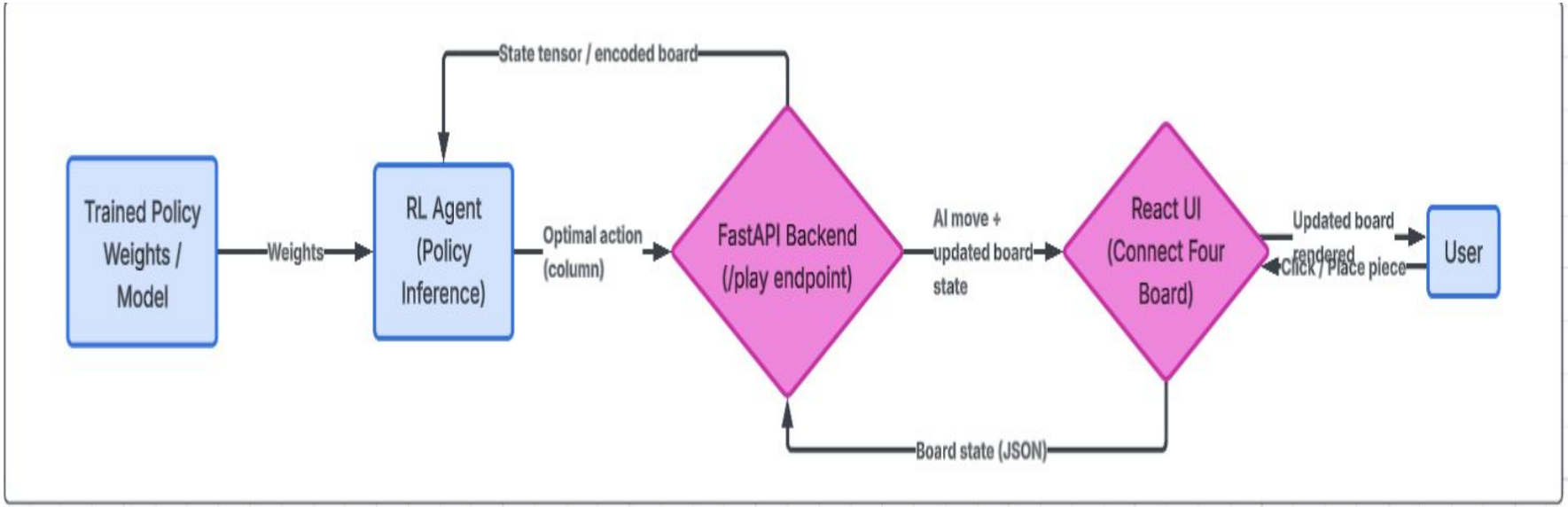
- **Frontend:** React/TypeScript handles the UI and Game Engine rendering.
- **Backend:** FastAPI acts as the bridge, managing game state and auth.
- **AI Engine:** PyTorch and Stable Baselines3 run the PPO/DQN models, while classical algorithms (Minimax) serve as a fallback.
- **Link:** [System Architecture]



Data Flow Diagram

This shows how data moves from a user's click to the AI's decision-making process, illustrating the real-time interaction between the Frontend, Backend, and AI Engine.

1. **Input:** User makes a move (e.g., placing a piece in Connect Four).
2. **Request:** React sends the board state to the FastAPI /play endpoint.
3. **Inference:** The backend passes the state to the RL Agent.
4. **Reward/Action:** The Agent returns the optimal move based on its trained policy.
5. **Response:** The UI updates with the AI's counter-move and the new board state.



Environment Setup: Gymnasium

- **The Challenge:** To train Reinforcement Learning agents, each puzzle game (Wordle, Sudoku, etc.) must be modeled as a standardized environment where the agent can observe the state and take discrete actions.
- **The Solution (Gymnasium):** I utilized the industry-standard **Gymnasium** (formerly OpenAI Gym) framework to create wrapper classes for each game. This standardized interface allows the PPO and DQN agents from Stable Baselines3 to be seamlessly plugged into the environment.
- **State Space Definition:** For games like Connect Four, the **observation space** is defined as the current board configuration (a numerical matrix). For a game like Wordle, the state includes the history of guesses and feedback.
- **Action State Definition:** The **action space** defines the valid moves an agent can make (e.g., placing a token in one of seven columns, or selecting a letter). This standardization was the critical first step for all subsequent AI development.

Trained Data & Experience Buffers

- **Experience Replay Buffers:** For the PPO/DQN agents, data consists of crucial state-action-reward-next-state transitions. These are stored in memory buffers to stabilize the neural network training process and break correlation in the data.
- **Real-time Environment State Data:** This data is generated dynamically as the user or AI plays, comprising game-state observations (like numerical board matrices) passed through the Gymnasium interface.
- **Heuristic Logic Sets:** These predefined rule sets, used for games like Connect Four and Sudoku, provide a consistent "Expert" baseline. This data is critical for benchmarking the performance of the complex RL agents.
- **Genetic Population Data:** Derived from specialized models (like the "Smart Rockets" example), this data uses a "DNA" pool of steering force vectors that evolve across generations based on fitness scores.

Sources:

1. Reinforcement Learning by Nicholas Renotte: Reinforcement Learning for Gaming
2. The Nature of Coding Example #92 by Daniel Shiffman: Example 92 Smart Rockets
3. The Nature of Coding Example #93 by Daniel Shiffman: Example 93 Smart Rockets

Hours of Learning: PyTorch & RL Stack

To effectively implement the advanced PPO and DQN models, significant self-directed learning was required, which forms a major component of the project's technical contribution.

- **Investment Estimate:** I estimate dedicating 70+ hours over the semester to deep-drive learning into core Reinforcement Learning concepts and the PyTorch/Stable Baselines3 framework.
- **Core Topics Learned:**

1. **Deep Q-Networks (DQN):** Understanding Q-tables, experience replay, and target networks for discrete action-space games (like Connect Four).
2. **Proximal Policy Optimization (PPO):** Learning policy gradients, clipping, and why this algorithm is preferred for more complex, continuous-state environments.
3. **Gymnasium Standardization:** Mastering the creation of custom game environments that adhere to the standard RL API, enabling seamless model training.

- **Key Resources Utilized:** The implementation was grounded in external research and tutorials from sources focused on practical RL application, including the work of Nicholad Renotte and Daniel Shiffman (The Nature of Code).

Algorithm Comparison: Classical vs. RL

Feature	Classical Algorithms (Heuristic Logic, Minimax)	Reinforcement Learning (PPO, DQN)
Core Approach	Pre-programmed rules and search techniques (e.g., Connect Four).	Policy-based or Value-based learning through thousands of environment interactions.
Implementation	Simple logic, faster setup, and low computational overhead.	Requires complex environment standardization (Gymnasium) and high training time.
Performance	Provides a strong deterministic, and predictable baseline for comparison.	Achieves near optimal performance; highly adaptable to complex, unseen game states.
Project Use Case	Used for providing the Expert Baseline for benchmarking against modern models.	Used for achieving Superhuman performance and demonstrating the power of machine learning in game theory.

Algorithm Deep Dive: PPO Implementation

- **Choice of PPO:** Proximal Policy Optimization (PPO) was selected as the flagship algorithm due to its excellent balance of sample efficiency, stability, and ease of implementation via Stable Baselines3.
- **Application:** PPO is a particularly effective for games with high state-space complexity (like the Connect Four environment), where rapid, aggressive learning updates must be avoided to ensure policy convergence.
- **Stability Mechanism:** PPO utilizes a clipping mechanism that restricts how far the new policy can deviate from the old policy during an update step. This prevents large, destabilizing changes, leading to smoother, more reliable training.
- **Architecture:** The agent operates with two neural networks: a Policy Network (chooses the action) and a Value Network (estimates the quality of the state). Both are trained simultaneously to achieve optimal performance.
- **Contribution:** The PPO agent represents the peak technical achievement of the platform, serving as the “Superhuman” opponent for users and a key performance indicator for benchmarking.

Critical Design Decision: Modular Architecture

- **The Design Goal:** Create a backend that allows dynamic swapping between different AI strategies (Classical, DQN, PPO) without downtime or major code changes. This is necessary for implementing the "Agent-vs-Agent Benchmarking" feature.
- **The Solution (FastAPI Bridge):** I developed a scalable, modular architecture using FastAPI. The backend acts as a **Strategy Pattern** wrapper, abstracting the AI implementation details.
- **Seamless Switching:** The API is designed so that a single call (e.g., /api/v1/play?agent=PPO vs. /api/v1/play?agent=Heuristic) selects the appropriate model from the AI Engine.
- **Benefit:** This modularity was crucial for iterative benchmarking and efficient debugging of agent-environment interactions. It confirms that the platform is built not just for playing, but for comparing and visualizing diverse AI methods.

Key Technical Challenge: Model Convergence

- **The Hurdle:** The most significant technical challenge was achieving stable and optimal performance (convergence) when integrating Reinforcement Learning (RL) agents, specifically PPO and DQN, into diverse game environments. RL models are highly sensitive to small changes, and improper configuration often leads to unstable learning or non-optimal results.
- **The Solution:** I conducted research and testing to strategically match specific game dynamics with the most effective AI strategy.

1. **PPO:** Utilized for continuous state-spacing challenges where stability is paramount.
2. **DQN:** Applied to discrete action-based environments (like Connect Four) where Q-value prediction is more straightforward.

- **Outcome:** This tailored approach stabilized the models and allowed the agents to consistently learn game-winning policies, directly validating the technical integration component of the project.

The Toughest Bug: The Ghost Move

- **The Scenario:** During initial testing of the Connect Four environment, the trained PPO agent would sometimes make a "ghost move"—an action that resulted in no change to the UI or game state, but still consumed a turn. This bug was intermittent and nearly impossible to reproduce consistently.
- **The Root Cause:** After extensive debugging, the issue was traced to a subtle flaw in the is_valid_action() check within the Gymnasium environment wrapper. The check was failing to correctly handle array slicing in the board state when a column was full, occasionally allowing the agent to select an illegal action (column 7) which the game logic then silently ignored.
- **The Obstacle:** This bug was challenging because it required tracing data flow through three layers: the **PPO Agent's Policy**, the **FastAPI Bridge**, and the **Gymnasium Environment**.
- **The Solution:** I implemented strict, dual validation: first within the agent's action mask to ensure only legal moves were possible, and second, a robust assertion within the Gymnasium wrapper's step() function to throw an immediate error if an illegal move was attempted. This isolated the failing logic and stabilized the environment.

Code Snapshot/Functionality

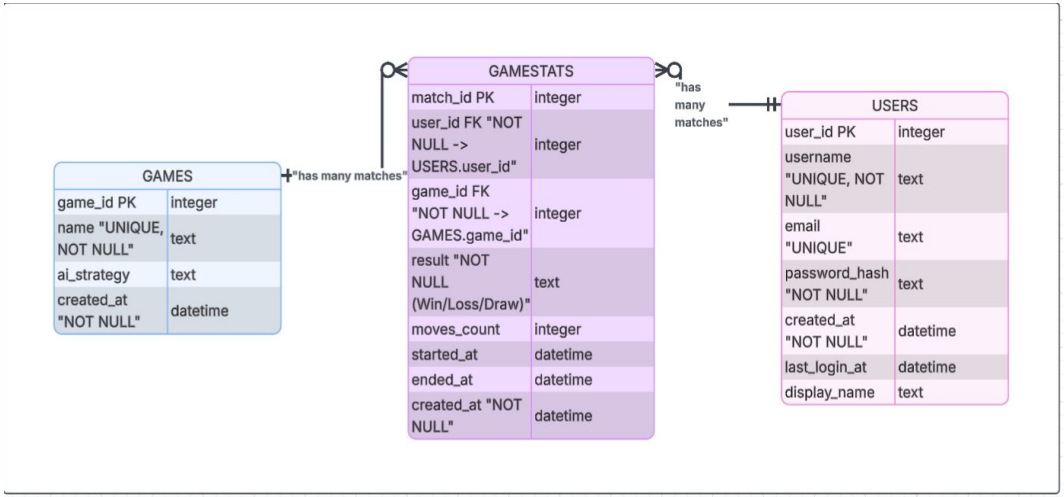
- This snippet demonstrates the core logic of the modular backend, which routes a user's game move to the appropriate AI agent (Heuristic or PPO) based on URL query parameter.

```
# FastAPI endpoint for processing a game move

@router.post("/play")
async def process_move(data: GameMoveSchema, agent_type: str = Query("PPO")):
    """Handles user input, selects AI, and returns the agent's counter-move."""

    # 1. Select the appropriate agent strategy dynamically
    if agent_type.upper() == "HEURISTIC":
        agent = HeuristicAgent()
    elif agent_type.upper() == "PPO":
        agent = PPOAgent()
    else:
        raise HTTPException(status_code=400, detail="Invalid agent type.")
    # 2. Update game state based on user input
    current_state = update_game_state(data.board, data.user_move)

    # 3. Get the optimal action from the selected AI
    ai_move = agent.predict_action(current_state)

    # 4. Return the new state to the React frontend
    return {"status": "success", "ai_move": ai_move, "new_state": new_board}
```

Database Schema (ERD)

This ERD (Entity Relationship Diagram) illustrates the persistent data structure, mapping out how the platform track users, their performance, and game context using PostgreSQL.

- **Users Table:** Stores user authentication and profile information, enabling personalized game statistics.
- **Games Table:** Categorizes the different environments (Chess, Othello, etc.) and links them to the available AI strategies (Heuristic, PPO, DQN).
- **GameStats Table:** This is the core transactional table, recording every match result, key performance indicators (KPIs) like total moves made, and timestamps for comprehensive performance tracking and benchmarking
- **Link**: SQLite Platform ERD



Value Proposition: Driving the Problem Space

- **Primary Value:** The core goal is to bring *Grumpy Gamer* to the public and have it be a functional, interactive **AI benchmarking tool** that everyone can use.
- **Driving Education Forward:** Because my project exists, students now have a hands-on platform to visualize complex Reinforcement Learning policies (PPO/DQN) and compare them directly against classical algorithms. This accelerates the practical understanding of machine learning strategies.
- **Support Development:** The modular and scalable backend architecture (FastAPI) provides a robust, standardized environment for rapidly testing and deploying *new* AI agents and game environments
- **Bridging Theory and Practice:** The platform serves as a critical bridge, transforming abstract computer science theory, such as model convergence and policy optimization into tangible, competitive, and enjoyable game scenarios.

Use Case 1: The Student Learner: AI Strategy Visualization

- **User:** A Computer Science student or AI enthusiast.
- **Scenario:** The user wants to understand how a Reinforcement Learning agent (PPO/DQN) differs from a classical algorithm (Minimax).
- **Action:** They play a game of Connect Four against the "Pro" agent (PPO) and then switch to the "Heuristic" agent.
- **Outcome:** Through the **Game Analytics** dashboard, the student visualizes the AI’s decision-making heatmaps and "confidence levels," gaining a practical understanding of machine learning theory.

Use Case 2: The Casual Gamer: The Ultimate “Personal” Coach

- **User:** Someone looking to improve their skills in complex games like Sudoku or potentially Chess.
- **Scenario:** The user is stuck on a difficult board or a mid-game position that needs strategic guidance.
- **Action:** They engage the **"Hint"** feature, which queries the backend AI agent to suggest the mathematically "optimal" move.
- **Outcome:** The AI provides the suggested move along with a brief, high-level explanation (e.g., "This move minimizes the opponent's future mobility"), helping the user learn better strategies while playing.

Use Case 3: The Developer: "Agent vs. Agent Benchmarking"

- **User:** A developer (like yourself) testing model performance.
- **Scenario:** You have just finished training a new DQN model for Othello and need to see how it performs against a standard heuristic.
- **Action:** You enter Spectator Mode, setting two AI agents to play 100 rounds against each other at high speed.
- **Outcome:** The system records win/loss ratios and move efficiency, allowing you to validate that the new RL model is actually outperforming the baseline code.

Use Case 4: The Competitive Player: "Beat the Unbeatable"

- **User:** A high-skill player looking for a challenge.
- **Scenario:** The user finds most mobile game AI too predictable and easy.
- **Action:** They select **"Grumpy Mode"** in 2048 or Minesweeper, which activates the most highly-trained RL agent.
- **Outcome:** The user gets a high-intensity session where the AI plays with near-perfect efficiency, pushing the user to improve their own tactical speed and accuracy.

Conclusion: My Journey & Key Takeaways

- **My Contribution:** I successfully developed, deployed, and validated the *Grumpy Gamer* platform, bridging the gap between complex AI theory and interactive environment.
- **Technical Achievement:** My hybrid approach seamlessly integrating advanced Reinforcement Learning (PPO/DQN) with classical algorithms via a FastAPI backend created a versatile and scalable system.
- **Learning Outcome:** I mastered the entire full-stack process, from environment setup (Gymnasium) and model training (PyTorch) to production deployment (Vercel/Render), while effectively managing scope and overcoming key technical hurdles like model convergence.
- **Project Value:** The final platform is a robust, publicly accessible tool that visualizes AI decision-making, offering clear, practical insights into machine learning strategies for both students and competitive players.

Future Work & Project Roadmap

- **Phase 1: Chess Environment Integration (Immediate):** Complete the initial goal of fully integrating the Chess environment (which was previously downscoped) and commence training a competitive RL agent using the established PPO framework.
- **Phase 2: Full Deployment Scaling:** Transition the current production deployment from Render/Vercel to a more robust, scalable cloud infrastructure (e.g., AWS or GCP) to handle increased user load and dedicated GPU resources for AI inference.
- **Phase 3: Interactive Training Feature:** Implement a user interface that allows users to contribute to the model's training by submitting their own game sessions as input data, accelerating agent improvement.
- **Phase 4: Advanced Metrics:** Expand the GameStats database to include more complex metrics, such as Action Entropy and Policy Divergence, which would offer deeper insights into the PPO agent's decision-making process.

Project Reflection: What I Learned

- **Mastering Scope Definition:** The decision to downscope the full Chess environment was the most valuable lesson in project management. I learned to aggressively prioritize the Minimum Viable Product (MVP) core, the benchmarking framework over the features.
- **Solo Project Management:** As a solo developer, I gained experience wearing multiple hats from backend architect to QA engineer. This required strict adherence to my self-imposed Agile schedule and constant context scheduling.
- **Debugging Complex Systems:** I learned that debugging Machine Learning systems (especially RL agents) is fundamentally different from traditional software. It requires careful systematic testing of the environment (Gymnasium) and the reward function, not just the code syntax.
- **The Importance of Infrastructure:** Investing early in a scalable, modular architecture (FastAPI) paid off by making agent switching and debugging far more efficient. Strong infrastructure minimizes future integration headaches.

Advice for Future CISC 4900 Students

- **Prioritize Environment Modeling Early:** Do not underestimate the time required to wrap game logic into the standardized Gymnasium interface. Until the environment is stable, you cannot begin training or debugging agents.
- **Aggressive Scope Management is Key:** Plan for 30% more time on the core technical hurdles (model training, debugging) than you initially estimate. If a feature is not critical to the MVP, defer it immediately to protect your deadline.
- **Validate the Reward Function First:** The most common failure point in a Reinforcement Learning project is an incorrectly defined reward function. Before training, manually test that the agent receives the intended reward for every possible action.
- **Utilize the Project Board:** Treat your Github Projects or timelog as mandatory daily checkpoint. Detailed record keeping is the only way to track velocity and demonstrate your contributions over the semester.

Acknowledgements

- **Academic Guidance:** Zack Desario (Supervisor/Academic Advisor) for providing essential conceptual guidance on Reinforcement Learning strategies and technical mentorship throughout the project timeline.
- **Technical Inspiration:** Nicholas Renotte, whose comprehensive tutorials on Reinforcement Learning were instrumental in implementing the PyTorch and Stable Baselines3 agents.
- **Modeling Frameworks:** Daniel Shiffman (The Nature of Code), whose foundational examples guided the genetic population data modeling and provided key insights into environment creation.

Final Links & Contact

Final Links & Contact

- Presenter: Faizan Khan
- Email: faizanakhan2003@gmail.com
- Linkedin: https://www.linkedin.com/in/faizan-khan234
- Github: https://github.com/jellyfishing2346

Project Access Links:

- Live Deployment (Hosted Project): https://grumpy-gamer.vercel.app/
- Source Code (Github Repository): https://github.com/jellyfishing2346/grumpy-gamer
- Project Management Board (Time Log/Agile): https://github.com/users/jellyfishing2346/projects/6